Software Requirements Specification (SRS)

Civic Issue Reporting and Resolution Platform

September 2025
Version 1.1

1 Introduction

1.1 Purpose

The purpose of this system is to provide a **mobile-first civic issue reporting platform** that enables citizens to submit real-world issues (e.g., potholes, garbage, broken streetlights) and allows municipal authorities to efficiently track, prioritize, and resolve these issues through an administrative portal.

1.2 Scope

The platform consists of:

- A **mobile application** for citizens to report and track issues.
- A **web-based administrative portal** for municipal staff.
- A **backend system** with automated routing, real-time updates, and analytics.

Expected benefits:

- Faster identification and resolution of civic issues.
- Increased transparency and accountability.
- Improved citizen engagement.
- Scalable architecture suitable for multiple cities.

1.3 Definitions, Acronyms, Abbreviations

- **Citizen App** – Mobile application for users to report and track civic issues.
- **Admin Portal** – Web-based system for municipal staff to manage reports.
- **Routing Engine** – Backend module that assigns issues to the appropriate department automatically.
- **GIS** – Geographic Information System for location-based services.
- **SRS** – Software Requirements Specification.

2 Overall Description

2.1 Product Perspective

The system bridges **citizens** and **municipal departments**, integrating real-time reporting with automated workflows and analytics.

2.2 Product Functions

- **Citizens:** Submit issues, track status, receive notifications.
- **Admins:** View, filter, categorize, assign, and resolve reports.
- **Backend:** Store reports, route tasks, provide APIs, and generate analytics.

2.3 User Classes and Characteristics

- **Citizens:** Smartphone users, casual or tech-savvy, requiring a simple and fast interface.
- **Municipal Staff (Admins):** Staff responsible for reviewing and resolving issues efficiently.
- **Super Admins:** Manage departments, configure rules, and monitor analytics.

2.4 Constraints

- Must be mobile-first and responsive across devices.
- Must handle high volumes of image/video uploads efficiently.
- Must ensure secure handling of all user data.

2.5 Assumptions and Dependencies

- Users have internet access and GPS-enabled devices.
- Municipalities provide necessary staff for processing reports.
- The system may integrate with future CRM or GIS systems.

3 Functional Requirements

3.1 Citizen Mobile Application

- **FR-1:** Users shall register/login using email, phone, or social login.
- **FR-2:** Users shall submit an issue report including:
  - One or more photos or a short video.
  - Automatic GPS location tagging.
  - Short text or voice note.

- **FR-3:** Users shall view a **live map** displaying all reported issues.
- **FR-4:** Users shall track report status with clear stages: **Submitted → Acknowledged → In Progress → Resolved**.
- **FR-5:** Users shall receive notifications at each stage (confirmation, acknowledgment, resolution).
- **FR-6:** Users shall optionally upvote or comment on existing reports to validate issues.

3.2 Admin Web Portal

- **FR-7:** Admins shall log in securely.
- **FR-8:** Admins shall view incoming reports on a dashboard with filtering by **category, location, and priority**.
- **FR-9:** Admins shall manually assign tasks to relevant staff when necessary.
- **FR-10:** The system shall automatically route issues to appropriate departments based on report metadata.
- **FR-11:** Admins shall update the status of reports.
- **FR-12:** Admins shall generate analytical reports (e.g., response time, trends, departmental efficiency).
- **FR-13:** Admins shall communicate status updates to citizens via notifications.
- **FR-14:** Admins shall define **configurable criteria** for issue prioritization (e.g., urgency, volume, category).

3.3 Backend System

- **FR-15:** The system shall store all reports, including images, videos, location, and metadata.
- **FR-16:** The system shall provide APIs for mobile and web clients.
- **FR-17:** The routing engine shall assign tasks automatically based on metadata (issue type, location, urgency).
- **FR-18:** The system shall support **concurrent requests** with low latency.
- **FR-19:** The system shall maintain an **audit log** for all activities.

4 Non-Functional Requirements

4.1 Performance

- **NFR-1:** The system shall handle at least **10,000 concurrent users**.
- **NFR-2:** Report submissions (including images/videos) shall complete within **5 seconds** under normal network conditions.
- **NFR-3:** Live map updates shall reflect new data within **2 seconds**.

4.2 Scalability

- **NFR-4:** Backend shall support **horizontal scaling** (cloud-based).
- **NFR-5:** System shall handle **10x traffic spikes** without downtime.

4.3 Security

- **NFR-6:** All communication shall use **HTTPS/TLS 1.3**.
- **NFR-7:** User data shall be encrypted at rest and in transit.
- **NFR-8:** Authentication shall use **OAuth2/JWT**.

4.4 Usability

- **NFR-9:** Citizen app shall follow **mobile-first UX principles**, ensuring simple workflows.
- **NFR-10:** Admin portal shall support **responsive design** for desktop and tablet.
- **NFR-11:** The system shall provide a **seamless cross-device experience**, allowing users to continue reports on different devices.

4.5 Reliability & Availability

- **NFR-12:** System uptime shall be **99.5%**.
- **NFR-13:** System shall **auto-recover** from server failures.

4.6 Maintainability

- **NFR-14:** Codebase shall follow **modular architecture** (microservices preferred).
- **NFR-15:** APIs shall be **versioned** for backward compatibility.

4.7 Analytics

- **NFR-16:** Dashboards shall provide:
  - Average resolution time.
  - Top issue categories.
  - Department efficiency.
- **NFR-17:** Data shall be exportable in **CSV and PDF formats**.

4.8 Upload & Data Handling

- **NFR-18:** The system shall maintain **explicit upload performance under peak load**.

5 System Models (Optional for Hackathon)

- **Use Case Diagram:** Citizen submits report → Routing engine → Department → Admin updates → Citizen notified.
- **Data Flow Diagram (DFD):** Inputs (photo, location, text) → Backend storage → Routing engine → Admin portal → Updates back to citizen app.
- **High-Level Architecture Diagram:** Mobile App Backend API Database & Routing Engine Admin Portal Analytics Module.